

Rank 1 Vulnerability Fix — Session Log

AES-128 Peripheral: Key Registers Made Write-Only

Waylon Woo with Claude Opus 4.6 (1M context)

2026-04-12

Session Log: Rank 1 Vulnerability Fix

Environment: Claude Code v2.1.104 (Opus 4.6, 1M context, high-effort mode) **Working directory:**

`~/repo/projects/AI-HDL/main` **Outcome:** Fix merged as commits `c2c553f` + `dde25ef` on branch `DP3`, pushed to `origin`.

Turn 1 — Read the security report

User: Can you read the security report in DP3_submission?

(Claude searched for the file, read the 13-page PDF.)

Claude: I read the 13-page AES-128 security evaluation report at `/Users/ww/repo/projects/AI-HDL/main/DP3_submission/security_report.pdf`. Quick summary:

Multi-agent evaluation (3 agents, 3 rounds, dated 2026-03-31) of an AES-128 Verilog engine for Tiny Tapeout (`tt_um_tqv_peripheral_harness`).

Ranked vulnerabilities:

1. **CRITICAL** — Key registers readable via bus (`0x02-0x05`); `key_lock` only blocks writes. ~10-line fix.
2. **CRITICAL** — FSM (2-bit) and round counter (5-bit) lack integrity protection; single glitch → round skipping / DFA.
3. **HIGH** — Unmasked S-boxes; DPA breaks it in 500–800 traces.
4. **HIGH** — Two-phase SubBytes creates asymmetric fault window ideal for DFA.
5. **HIGH** — Soft reset race condition (DoS + encryption with partially-zeroed key).
6. **MEDIUM** — Fixed 22-cycle timing eases trace alignment.
7. **MEDIUM** — No fault detection / error signaling.
8. **LOW** — No FIPS-197 known-answer self-test.

Immediate pre-tapeout fix: make key registers write-only + add busy interlock on soft reset. Unresolved constraints: area budget for masking, no TRNG on Tiny Tapeout, no bus privilege model, encryption-only (ECB-prone).

Turn 2 — List specific ranks

User: List the vulnerabilities rank 1, 2, 5, 7

Claude:

Rank 1 — CRITICAL: Key Registers Readable via Bus. Key stored at addresses `0x02–0x05` is readable by any software. `key_lock` only prevents writes, not reads. Four bus reads recover the full key — no equipment, no skill needed. Fix: return zeros on read (~10 lines RTL).

Rank 2 — CRITICAL: FSM and Round Counter Have No Integrity Protection. 2-bit FSM and 5-bit round counter with no error checking. A single voltage glitch (~\$200 ChipWhisperer) can flip COUNTING → DONE (early finish → DFA), COUNTING → IDLE (silent abort), or bump counter 19 → 21 (round skip). Full key recovery with 2–4 glitches. Fix: one-hot FSM + shadow counter.

Rank 5 — HIGH: Soft Reset Race Condition. Control register bit 1 clears everything, callable by any software mid-encryption. Enables DoS (wipe key on multi-tenant systems) and “encryption with a dead key” where mid-expansion reset mixes original-key and zero-key round keys. Fix: busy interlock + magic-value two-step reset.

Rank 7 — MEDIUM: No Fault Detection or Error Signaling. `done` fires whether encryption succeeded or was corrupted; `data_ready` is hardwired to 1. Every fault-injection attempt yields usable ciphertext for DFA. Fix: add `fault_detected` pin that outputs zeros and requires reset on illegal FSM state / out-of-range counter / redundancy mismatch.

Turn 3 — Pick a vulnerability; scope the work

User: Let’s work on Rank 1 vulnerability. How might we attack that problem?

(Claude invoked the `superpowers:brainstorming` skill and read `AES_memory.v`.)

Claude: I’ve read the code at `AES_memory.v:147–150` — the key registers are directly exposed in the read-path `case` statement.

Before I propose approaches, one clarifying question: **what’s the scope of this work?**

- **A.** Just the Rank 1 fix — minimal RTL change, land it and move on.
- **B.** Rank 1 + tidy the read path so future additions don’t accidentally re-expose the key (defensive structure).
- **C.** A broader revision that bundles Rank 1 with the other “immediate pre-tapeout” fix (Rank 5 soft-reset interlock) into one pass.

Which fits what you’re trying to do?

User: A

Claude: Three ways to attack the read path at `AES_memory.v:147–150` :

- **A.** Delete the cases — remove `0x02–0x05` entries; they fall through to the existing `default: 32'b0` . Smallest diff (4 lines removed).
- **B.** Explicit zero with comment — keep cases, assign `32'b0` , add `// write-only: key not readable` comment. What the report recommended.
- **C.** Return a sentinel (e.g. `32'hBADC0DE5`) — helps firmware detect buggy reads during debug.

Recommendation: B. Documents the security property at the point of enforcement — a future reader sees “this is deliberately zero” instead of wondering whether the address was forgotten. Cost is identical to A in gates (synthesizes the same).

Does B work for you, or want to discuss?

Turn 4 — Explain the plan in detail

User: Explain option B in detail. What are we changing and why?

Claude:

What we're changing

File: `/Users/ww/repo/projects/AI-HDL/main/AES128/AES_memory.v`

In the combinational read block (lines 141–169), we modify four case entries that currently expose the key.

Before (lines 147–150):

```
6'h02: data_out = key_mem[0];
6'h03: data_out = key_mem[1];
6'h04: data_out = key_mem[2];
6'h05: data_out = key_mem[3];
```

After:

```
// Key registers are write-only. Reads return zero so that a
// compromised or buggy master on the bus cannot exfiltrate the
// secret key. key_lock (control_reg[2]) prevents *overwrites*;
// this clause prevents *reads*. Both are required.
6'h02: data_out = 32'b0;
6'h03: data_out = 32'b0;
6'h04: data_out = 32'b0;
6'h05: data_out = 32'b0;
```

Nothing else changes. The write path at lines 111–115 still accepts writes to `0x02–0x05` (gated by `key_lock`). The `original_key` wire at line 59 still feeds the key engine internally. Only the bus-visible read value changes.

Why each piece is the way it is

Why not delete the cases entirely (option A)? A future engineer adding a new register might look at the read path, see `0x02–0x05` missing, assume it's an oversight, and re-add `data_out = key_mem[0]` “for symmetry with the write path.” The explicit `32'b0` + comment blocks that mistake at the exact line where it would be made. The synthesizer produces identical gates either way.

Why `32'b0` rather than a sentinel like `32'hDEADBEEF`?

- Zero matches the module's existing `default` return, so the bus behavior is uniform and predictable.
- Zero is what a reset register would naturally return; firmware written under the (wrong) assumption that keys are readable will see “key is all zeros” and fail loudly rather than silently consume garbage.
- A sentinel would require firmware to know the magic value, adding coupling. Not worth it for a minimal fix.

Why keep the cases rather than letting them fall through to default? Pure readability. When someone greps for `6'h02` to understand what's at that address, they find it explicitly handled with a comment explaining the security property.

Why the comment mentions `key_lock`? The report's core insight is that `key_lock` is a *write* protection and was mistakenly assumed to also block reads. The comment makes the write/read distinction explicit so the next reader doesn't repeat the confusion.

What this does NOT change

- Write behavior: software can still load keys at `0x02–0x05` when `key_lock == 0`.
- Internal key propagation: `original_key` still drives the key engine.
- Any other address, including control (`0x00`), status (`0x01`), plaintext banks (`0x06–0x09`), or ciphertext output (`0x0A–0x0D`).
- Timing, area, or the FSM. No new flip-flops, no new logic — just a constant driver in place of the `key_mem[]` tap.

Threat model coverage

After this change:

- **Defeated:** software-only key extraction via bus reads (Rank 1 as described).
- **Not defeated:** physical side-channel attacks (Ranks 3, 4, 6), fault injection (Ranks 2, 4, 7), soft-reset race (Rank 5).

Verification

A single directed test suffices: write a known nonzero key to `0x02–0x05`, read each address back, assert the readback is `32'h00000000`. Optionally also assert that encryption still produces the correct ciphertext for a FIPS-197 test vector — proving the key still reaches the engine internally even though it's invisible on the bus.

Turn 5 — Run the tests first

User: Before we implement this change, can we run our tests?

(Claude read `AES_memory_tb.v`.)

Claude: Important finding before we run anything — looking at `AES_memory_tb.v:133–136, 163, 270–273, 283`, the existing testbench explicitly asserts that key reads return the written values:

```
write32(6'h02, 32'hAAAAAAAA);  
...  
check32(6'h02, 32'hAAAAAAAA); // expects readback = written value
```

So Option B will break these checks by design — they encode the vulnerable behavior as a contract. The fix therefore has two parts: RTL change + test update (flip those `check32` calls to expect `32'h0`).

Let me first establish the baseline.

(Claude checked for installed simulators.)

```
iverilog not found  
vvp not found  
verilator not found  
/opt/homebrew/bin/verible-verilog-lint (syntax only, not a simulator)
```

Claude: I can't run the tests — no simulator is installed. Three ways forward:

- **A.** Install Icarus (`brew install icarus-verilog`) and run it.
- **B.** Skip the sim, reason from the testbench source. 10 `check32` lines will flip from PASS to FAIL after Option B.
- **C.** Do both — install + run baseline, then change RTL + tests together, rerun.

I'd recommend A before touching anything. Which do you want?

Turn 6 — Environment issue #1

User:

```
> python3 test.py  
ModuleNotFoundError: No module named 'cocotb'
```

(Claude inspected `requirements.txt` and `test.py`.)

Claude: Two things missing:

1. **cocotb** — `requirements.txt` pins `cocotb==1.9.2` but it's not installed in system Python, and there's no venv.

2. iverilog — not installed.

To run from a clean baseline:

```
cd /Users/ww/repo/projects/AI-HDL/main
brew install icarus-verilog
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
make                # runs cocotb + iverilog on test.py / tb.v
```

And for the AES-specific directed testbench:

```
cd AES128/testbenches
iverilog -o aes_mem_tb AES_memory_tb.v ../AES_memory.v && vvp aes_mem_tb
```

Do you want me to run those setup commands? `brew install` and `pip install` modify your environment, so I'd rather ask before touching them.

Turn 7 — Environment issue #2: declaration-order bug

User: (after installing iverilog + cocotb)

```
> python3 run_tb.py

=== Select a Testbench to Run ===
[1] Top Level Integration (AES_peripheral_tb)
...
Enter choice: 1

[ERROR] Compilation Failed:
AES128/AES_encryption_engine.v:11: error: Unable to bind wire/reg/memory `state_reg`
      in `tb_tqvp_example.dut.aes_encryption_engine_inst`
AES128/AES_encryption_engine.v:13:      : A symbol with that name was declared here.
      Check for declaration after use.
2 error(s) during elaboration.
```

Claude: That's a pre-existing bug unrelated to our Rank 1 work — `AES_encryption_engine.v:11` uses `state_reg` before it's declared on line 13. Icarus is stricter than some sims about declaration-before-use.

Two observations:

1. The bug blocks testbenches 1 and 3 (anything that includes the encryption engine). It does *not* block testbench 5 (Memory Interface) — which is the one that actually exercises the key read path we're changing.
2. Fixing that bug is trivial (swap line 11 and line 13), but it's out of scope for "A. Just the Rank 1 fix."

Can you run choice 5 and paste the output?

User: Can you fix this?

(Claude edited `AES_encryption_engine.v`.)

Diff:

```
- // cipher output is just state_reg - memory latches it on 'done'
- assign cipher = state_reg;
-
  reg [127:0] state_reg;
  reg [63:0] sub_out_hi;

+ // cipher output is just state_reg - memory latches it on 'done'
+ assign cipher = state_reg;
```

Claude: Done. Pure reordering — no logic change.

Turn 8 — Apply the Rank 1 fix

User: Make the changes for Rank 1 now.

(Claude edited `AES_memory.v`.)

Diff:

```
- 6'h02: data_out = key_mem[0];
- 6'h03: data_out = key_mem[1];
- 6'h04: data_out = key_mem[2];
- 6'h05: data_out = key_mem[3];
+ // Key registers are write-only. Reads return zero so that a
+ // compromised or buggy bus master cannot exfiltrate the secret
+ // key. key_lock (control_reg[2]) prevents *overwrites*; this
+ // clause prevents *reads*. Both are required – see security
+ // report Rank 1 (DP3_submission/security_report.pdf).
+ 6'h02: data_out = 32'b0;
+ 6'h03: data_out = 32'b0;
+ 6'h04: data_out = 32'b0;
+ 6'h05: data_out = 32'b0;
```

Claude: Now updating the testbench — some `check32` calls on key addresses tested security properties (`key_lock` blocked overwrite; key persisted across encryptions) via bus readback. Since readback now always returns 0, those properties need to be verified against the internal `original_key` wire instead, otherwise the test silently stops proving them.

(Claude attempted to edit the testbench; user rejected the edit and asked for an explanation.)

Turn 9 — Why the testbench must also change

Claude: Fair — I jumped ahead. Here's why the testbench needs changing:

Why the testbench must change

Our RTL change makes reads of `0x02-0x05` return `0`. But the existing testbench has 10 assertions that expect those reads to return the written key value. If we only change the RTL, those 10 assertions go from PASS to FAIL — the test suite reports “BROKEN” even though the security fix is correct.

Why just flipping expected values to `0` isn't enough

Three of those assertions were doing double duty: besides checking readback, they were the *only* way the test verified two real security properties:

1. `AES_memory_tb.v:163` — “After writing with `key_lock=1`, the key wasn't overwritten.” The test proved this by reading back and seeing the original key. If we change the expected value to `0`, the test will pass whether `key_lock` works or is completely broken — because reads return `0` unconditionally now.
2. `AES_memory_tb.v:270-273, 283` — “The key persists across encryptions.” Same problem: readback was the evidence. Flipping to `0` removes the evidence.

So we'd silently lose coverage of `key_lock` and key persistence. Those are exactly the kinds of properties you *don't* want to stop testing.

What `check_key` does

It peeks at the `original_key` wire (already routed to the testbench via the DUT port at `AES_memory_tb.v:14, 32`) and asserts it equals an expected 128-bit value. That wire is the concatenation of `key_mem[3:0]`. This lets the 3 security-property assertions keep testing what they were meant to test, just through a different observation point. The other 7 assertions are pure bus-readback checks and will simply become `check32(..., 32'h0)`.

Alternative

Skip `check_key`, accept the coverage loss. Lower code change, weaker tests. Not recommended.

Want me to proceed with `check_key`, or go with the minimal flip-to-zero alternative?

User: Go ahead and make those changes then.

Turn 10 — Apply the testbench updates

(Claude made four edits to `AES_memory_tb.v`.)

1. Added the `check_key` helper:

```

// Key is write-only on the bus (Rank 1 fix): reads of 0x02-0x05 return 0.
// Verify key-related security properties (key_lock, key persistence)
// against the internal original_key wire instead of bus readback.
task check_key(input [127:0] expected);
begin
    #1;
    if (original_key != expected) begin
        $display(" FAIL original_key: got 0x%032h expected 0x%032h", original_key, expected);
        fail_count = fail_count + 1;
    end else begin
        $display(" PASS original_key => 0x%032h", original_key);
        pass_count = pass_count + 1;
    end
end
endtask

```

2. After initial key write — flip readbacks to 0, add check_key :

```

- check32(6'h02, 32'hAAAAAAAA);
- check32(6'h03, 32'hBBBBBBBB);
- check32(6'h04, 32'hCCCCCCCC);
- check32(6'h05, 32'hDDDDDDDD);
+ // Bus reads of key addresses must return 0 (write-only key).
+ check32(6'h02, 32'h00000000);
+ check32(6'h03, 32'h00000000);
+ check32(6'h04, 32'h00000000);
+ check32(6'h05, 32'h00000000);
+ // But the key must have actually landed in the internal register.
+ check_key({32'hDDDDDDDD, 32'hCCCCCCCC, 32'hBBBBBBBB, 32'hAAAAAAAA});

```

3. key_lock overwrite check — verify the key wasn't mutated:

```

- // Key lock
- check32(6'h02, 32'hAAAAAAAA);
+ // Key lock: attempted overwrite must be ignored. Bus readback is 0
+ // regardless, so verify via original_key that the key wasn't mutated.
+ check32(6'h02, 32'h00000000);
+ check_key({32'hDDDDDDDD, 32'hCCCCCCCC, 32'hBBBBBBBB, 32'hAAAAAAAA});

```

4. Key-reuse test — verify key persists internally:

```

- // Verify key is still there
- check32(6'h02, 32'h2B7E1516);
- check32(6'h03, 32'h28AED2A6);
- check32(6'h04, 32'hABF71588);
- check32(6'h05, 32'h09CF4F3C);
+ // Verify key is still loaded internally (bus reads return 0 by design).
+ check32(6'h02, 32'h00000000);
+ check32(6'h03, 32'h00000000);
+ check32(6'h04, 32'h00000000);
+ check32(6'h05, 32'h00000000);
+ check_key({32'h09CF4F3C, 32'hABF71588, 32'h28AED2A6, 32'h2B7E1516});

```

Claude: Please run `python3 run_tb.py` → choice [5] (Memory Interface) and paste the output — I want to see a clean PASSED count before we consider this done.

Turn 11 — Memory TB passes, top-level fails

User: The memory interface tests pass but 1 test in top_level_integration does —

```
--- TEST 7: Soft Reset clears state ---
FAIL key written before reset
PASS key cleared after reset
PASS control reg cleared after reset

PASSED: 8
FAILED: 1
SOME TESTS FAILED
```

(Claude searched the peripheral TB.)

Claude: Two changes in that block. Reporting one failure but I'm proposing to change both — let me explain why.

Change 1: “key written before reset” — REQUIRED to fix the failure

Current code:

```
load_key;
bus_read(6'h02, tmp);
if (tmp === TEST_KEY[31:0]) ...PASS
```

Why it's failing: Our Rank 1 fix makes `bus_read(6'h02)` always return `0`. It will never match `TEST_KEY[31:0]`. This is the FAIL you're seeing.

What the check was for: proving `load_key` actually worked — so the subsequent “reset clears the key” check is meaningful. (If the key was never loaded, “cleared” is trivially true.)

Fix: peek at `dut.aes_memory_inst.original_key` (the internal `{key_mem[3:0]}` concat). That's the ground truth the bus read was trying to observe.

Change 2: “key cleared after reset” — currently passes, but for the wrong reason

Current code:

```
do_soft_reset;
bus_read(6'h02, tmp);
if (tmp === 32'h0) ...PASS
```

Why this is a problem: after our fix, `bus_read(6'h02)` returns `0` unconditionally. This assertion now passes even if `do_soft_reset` does absolutely nothing — the key registers could be full of the original key and the test would still report PASS. We silently lost coverage of the “reset actually zeroizes the key” property.

Fix: check `dut.aes_memory_inst.original_key === 128'h0` instead.

Approve both, or just change 1?

User: Approve both

(Claude edited `AES_peripheral_tb.v`.)

```
    load_key;
-   bus_read(6'h02, tmp);
-   if (tmp === TEST_KEY[31:0]) begin
+   // Key registers are write-only on the bus (Rank 1 fix): bus reads of
+   // 0x02-0x05 always return 0. Verify the key actually landed in the
+   // internal register by peeking at original_key.
+   if (dut.aes_memory_inst.original_key === TEST_KEY) begin
        $display(" PASS key written before reset");
        pass_count = pass_count + 1;
    end else begin
-       $display(" FAIL key written before reset");
+       $display(" FAIL key written before reset: got 0x%032h", dut.aes_memory_inst.original_key);
        fail_count = fail_count + 1;
    end

    do_soft_reset;

-   bus_read(6'h02, tmp);
-   if (tmp === 32'h0) begin
+   // After soft reset the internal key register itself must be zero -
+   // bus readback is 0 unconditionally now, so that's not honest evidence.
+   if (dut.aes_memory_inst.original_key === 128'h0) begin
        $display(" PASS key cleared after reset");
        pass_count = pass_count + 1;
    end else begin
-       $display(" FAIL key not cleared: 0x%08h", tmp);
+       $display(" FAIL key not cleared: 0x%032h", dut.aes_memory_inst.original_key);
        fail_count = fail_count + 1;
    end
end
```

Turn 12 — Race condition

User:

```
FAIL key written before reset: got 0x0000000028aed2a6abf7158809cf4f3c
```

Still getting this.

Claude: Found it. This is a race, not a test-coverage issue. `bus_write` finishes at the same `posedge` that commits the write (via NBA), and we're reading `original_key` *before* the NBA region closes — so `key_mem[3]` still shows the pre-write value (0 from the previous soft reset).

The earlier 3 words appear correct because their NBAs committed in earlier clock cycles. Only the most recent write (to 0x05) hasn't settled yet.

Fix: wait one cycle after `load_key` so the final NBA commits before we sample.

```
load_key;
+ // Wait a cycle so the NBA from the final bus_write (to 0x05) commits
+ // into key_mem[3] before we sample original_key.
+ @(posedge clk); #1;
// Key registers are write-only on the bus (Rank 1 fix): bus reads of
```

Claude: Rerun `run_tb.py` → [1] . Should show 9 PASS.

(User confirmed all tests pass.)

Turn 13 — Commit

User: Git add and commit the changes made to fix Rank 1 issue.

(Claude checked `git status` — found 6 modified files and 3 untracked paths.)

```
Changes not staged for commit:
  modified:   .gitignore
  modified:   AES128/AES_encryption_engine.v
  modified:   AES128/AES_memory.v
  modified:   AES128/testbenches/AES_memory_tb.v
  modified:   AES128/testbenches/AES_peripheral_tb.v
  modified:   test.py

Untracked files:
  DP3_submission/
  aes_memory_tb.vcd
  aes_test.vcd
  outputs/
```

Claude: There are more modified files than just the Rank 1 fix. Let me stage only the Rank 1 work. The `AES_encryption_engine.v` fix (declaration order) is separate — it was just needed to compile tests. I'll commit that separately after.

(Claude proposed staging four files; user requested `.gitignore` also include `*.vcd` and `outputs/`.)

.gitignore update:

```
runs/
.DS_Store
+ *.vcd
+ outputs/
```

(Claude staged `AES_memory.v`, both testbenches, and `.gitignore`, then attempted to commit. User interrupted to ask about the remaining unstaged changes.)

User: What changes in encryption engine and test.py?

Claude: Two unstaged changes:

- `AES_encryption_engine.v` — the declaration-order fix from earlier. Pure reordering, no logic change.
- `test.py` — mode change only (`644` → `755` , added executable bit). File contents unchanged. I didn't do this deliberately — probably `chmod +x` happened in your shell.

Suggestions: commit the encryption-engine fix separately; either revert the `test.py` mode or commit it separately.

User: Okay commit the Rank 1 fixes with a detailed git message.

(Claude committed as `c2c553f`. 5 files — `DP3_submission/security_report.pdf` was auto-added despite not being explicitly staged, which Claude flagged.)

User: git add the encryption engine changes and commit

(Claude committed `AES_encryption_engine.v` as `dde25ef`.)

Two clean commits on `main`:

- `c2c553f` — Fix Rank 1 vulnerability: make AES key registers write-only
- `dde25ef` — Fix declaration order in `AES_encryption_engine` so Icarus can elaborate

Turn 14 — Branch + push

User: make new branch called “DP3”

```
$ git checkout -b DP3
Switched to a new branch 'DP3'
```

User: push commits to this branch

```
$ git push -u origin DP3
remote: Create a pull request for 'DP3' on GitHub by visiting:
remote:      https://github.com/parsamrf/NECRL---AI-HDL/pull/new/DP3
* [new branch]      DP3 -> DP3
branch 'DP3' set up to track 'origin/DP3'.
```

PR URL: <https://github.com/parsamrf/NECRL---AI-HDL/pull/new/DP3>

Appendix A — Final File Inventory

File	Change	Purpose
<code>AES128/AES_memory.v</code>	4 key-read cases drive <code>32'b0</code> + comment	Close Rank 1 attack surface
<code>AES128/testbenches/AES_memory_tb.v</code>	Added <code>check_key</code> ; 10 readbacks → <code>0</code> ; 4 security- property verifications via <code>original_key</code>	Test passes on new RTL; security properties still verified
<code>AES128/testbenches/AES_peripheral_tb.v</code>	TEST 7 peeks <code>original_key</code> ; one-cycle settling delay after <code>load_key</code>	Top-level TB passes; reset- zeroization honestly verified
<code>AES128/AES_encryption_engine.v</code>	<code>reg</code> decls moved above <code>assign cipher =</code> <code>state_reg;</code>	Icarus compile-time fix
<code>.gitignore</code>	<code>*.vcd</code> , <code>outputs/</code>	Exclude simulator artifacts

Appendix B — Out of Scope

Ranks 2, 3, 4, 5, 6, 7, 8 from the security report remain unaddressed in this session. Each warrants its own scoped session; several (3, 4) may be infeasible on the Tiny Tapeout area budget.

Rank	Item	Status
2	FSM / round counter integrity	Open
3	Unmasked S-boxes (DPA)	Open
4	Two-phase SubBytes fault window	Open
5	Soft-reset busy interlock	Open
6	Timing jitter for DPA alignment	Open
7	<code>fault_detected</code> output	Open
8	FIPS-197 known-answer self-test	Open